Part 3 — Production Readiness Reflection

Q1. Evals and graders for the voice agent

I'd run three layers, all gating deploy. None of this is hypothetical — I built the equivalent for an LLM gateway at Zof, where a bad release hit real traffic.

Layer 1 — scenario replay. A fixed set of ~80 synthetic call transcripts: the four required flows plus the failure modes that actually bite. The emergency case appears in five phrasings ("severe chest pain and my arm is numb," "I think it's a heart attack," "my husband can't breathe," etc.), the "downtown" ambiguity appears with both offices, and there are partial-information callers, out-of-network insurance, and a caregiver who isn't an authorized proxy. Each transcript is scored by an LLM-as-judge (Claude Opus) against a per-scenario rubric. A failing test is any rubric item below 0.9 — e.g., the emergency turn that doesn't open with the 911 advisory, or a voicemail transcript that contains a patient name.

Layer 2 — function-call correctness. Hard pass/fail assertions on the tool trace, independent of phrasing. The emergency flow must never call `book_appointment`. The booking flow must call `verify_insurance` before `book_appointment` for new patients. A new appointment must never land on a provider/day the schedule doesn't allow (Dr. Webb in Oakland should be impossible). These catch regressions the language grader can miss.

Layer 3 — safety/PHI filters. A dedicated grader over every outbound voicemail transcript that flags any patient name, DOB, or clinical detail — the non-negotiable from the KB. Plus a grounding check: compare every factual claim the agent makes (a slot, an address, a co-pay) against the tool response that produced it, and flag anything the agent stated that no tool returned.

A failing test blocks deploy. A flaky test is treated as a fail and investigated, not muted. In production I'd sample ~5% of live calls daily through the same grader stack and review weekly, so drift surfaces before a patient does.

Q2. Prompt caching

OCR pipeline (where I applied it). The extraction rules and the per-type tool schema are identical for every document of a type; the page image is the only

variable input. So both carry `cache_control: {"type": "ephemeral"}` and sit at the front of the request (`extract.py`). Across a batch — or the three documents in this exercise — the second and third reads hit the cache instead of reprocessing the schema. The cached prefix is the large, stable majority of the input; the expected effect is roughly a 90% cost reduction on those cached input tokens, with the cache write costing ~25% extra on first use.

Voice agent (bigger latency win). The system prompt here is large — the full KB, the safety overrides, the PHI rules — and identical on every turn of every call. Cache it at the boundary before the live conversation turns. The mechanism is the same `cache_control` marker; the payoff is different. In voice the ~85% reduction in time-to-first-token on the cached prefix matters more than the cost — it shows up as the agent starting to speak sooner, which is the difference between feeling natural and feeling like an IVR.

Q3. Tooling split

Not preference — task fit.

Claude Code → **the OCR pipeline.** It's well-specified, file-system-heavy, and benefits from an autonomous build-test-fix loop. I'd hand it the spec and the three sample faxes, let it scaffold the repo, write the offline logic tests, and iterate until `pytest` is green — exactly the deny-back and lab-range tests that don't need an API key. That's the work where autonomy pays off.

Cursor → **the prompts.** The Retell system prompt and the extraction instructions are where I want a tight loop with every diff visible: change one rule, re-run a test call, watch the effect. Per-edit steering beats full autonomy when a single wrong line in the safety section is a patient-safety bug.

Codex (or a second agent) → **triangulation.** For the decisions I'm unsure about — how to structure confidence scoring, whether to split OCR from extraction — I'd give the same spec to a second model and compare proposals before committing. Cheap insurance against building the wrong thing well.

The principle: autonomy for well-specified subsystems, tight human-in-the-loop for the prompts that carry safety logic, and a second opinion where the architecture is still genuinely uncertain.